News Ingestion & Crawler Module

For this module, your objective is to build an **automated news ingestion pipeline** that can collect, evaluate, deduplicate, rank, and publish the most important AI/business news on our platform every day.

This module is different from the other modules because the primary challenge is **source discovery + automated data ingestion + news quality selection**.

1. Identify Reliable News Sources

First, create a list of approximately **100–500 high-quality English-language sources** from which we can collect news.

Sources can include:

- Official company blogs/newsrooms — OpenAI, Google, Microsoft, Meta, NVIDIA, etc.
- Trusted technology publications — TechCrunch, VentureBeat, The Verge, Wired, etc.
- Major global newspapers/business publications — Financial Times, New York Times, Washington Post, Bloomberg, Reuters, etc.
- Other credible AI, technology, startup, finance, and business publications.

Do not focus only on the number of sources. Quality and credibility are more important.

Create a structured source list containing the source name, URL, category, RSS/API availability, and crawling method.

2. Build the News Crawler

Build a pipeline that automatically checks these sources **at least once or twice every day**.

The pipeline should:

Source → Crawl/Fetch → Extract → Clean → Normalize → Store → Deduplicate → Rank → Publish

Where possible, use:

- RSS feeds
- APIs
- Official feeds
- Sitemap/news pages
- Website crawling/scraping

The goal is to minimize manual intervention.

3. Extract Structured News Data

For every article, extract and store information such as per our db table

The data must be stored in the format required by our existing website/database.

4. Do NOT Ingest Everything

This is extremely important.

We **do not want thousands of articles every day**.

Our target is approximately:

20–30 high-quality news stories per day across all sources.

The system should discover many articles but ultimately select only the **best and most relevant 20–30 stories**.

5. Build Duplicate Detection

The same news will often appear across multiple sources.

For example:

OpenAI launches a new product.

TechCrunch, Reuters, Bloomberg, The Verge, and several other publications may cover the same event.

The system should identify these as the **same underlying news event** and avoid publishing five separate copies.

Build logic for:

- URL duplication
- Exact/similar headline detection
- Semantic similarity
- Entity/event matching
- Multiple articles covering the same event

Where multiple sources cover the same event, we should ideally retain the **best/most authoritative coverage** while recording other sources where useful.

6. Build a News Importance / Ranking System

The biggest part of this module is determining:

Which 20–30 stories are actually worth publishing?

Create a scoring/ranking system.

Potential signals include:

Source credibility

- Is the source highly trusted?
- Is it an official company source?

Coverage

- Are multiple credible publications reporting the same event?
- How many independent sources are covering it?

Company importance

- Is the story about OpenAI, Google, Microsoft, NVIDIA, Meta, Anthropic, etc.?

Impact

- Does it represent a significant product launch, funding event, acquisition, research breakthrough, regulation, leadership change, business event, etc.?

Trending

- Is the story being discussed widely?
- Is there unusually high coverage or attention?

Recency

- How recently did the event happen?

Relevance

- How relevant is it to our AI/business audience?

The system should combine these signals into a **news importance score** and rank stories accordingly.

7. Handle Multiple Sources Intelligently

If 10 trusted sources report the same event, that is a strong signal that the event is important.

However, **number of articles alone should not determine importance**.

For example:

- 10 low-quality websites reporting a minor story ≠ important news.
- 3 highly credible sources reporting a major AI announcement = potentially very important news.

The ranking system should therefore consider **source quality + coverage + impact + relevance + recency**.

8. Automated Daily Pipeline

The final system should ideally run automatically every day:

1. Crawl sources

↓

2. Collect new articles

↓

3. Extract structured information

↓

4. Remove irrelevant articles

↓

5. Detect duplicate stories

↓

6. Group articles covering the same event

↓

7. Score/rank the stories

↓

8. Select the top 20–30

↓

9. Generate the required format

↓

10. Store/update the database

↓

11. Publish/update the website

There should be **minimal or zero manual intervention** in the normal workflow.

9. Fallback if Full Automation Is Not Possible

Your first objective should be **full automation**.

If certain sources cannot technically be crawled automatically because of:

- robots.txt restrictions
- authentication
- anti-bot systems
- API limitations
- technical restrictions
- legal/licensing limitations

then identify those limitations clearly and build the best possible alternative using RSS, APIs, feeds, or other permitted methods.

Only as a last resort should those sources require manual ingestion.

10. Expected Outcome

At the end of this module, we should have a reliable system that can answer:

“What are the 20–30 most important AI/business news stories today?”

without someone manually searching hundreds of websites every morning.

The system should be:

Automated + Reliable + Deduplicated + Ranked + Scalable + Easy to Monitor

Do not optimize for the **maximum number of articles**.

Optimize for the **maximum quality of the 20–30 articles we publish every day**.